



The University of Manchester

An Introduction to Gaussian Process Regression (GPR)

AERO40041

Data-driven Methods for Engineers

Saleh Rezaeiravesh

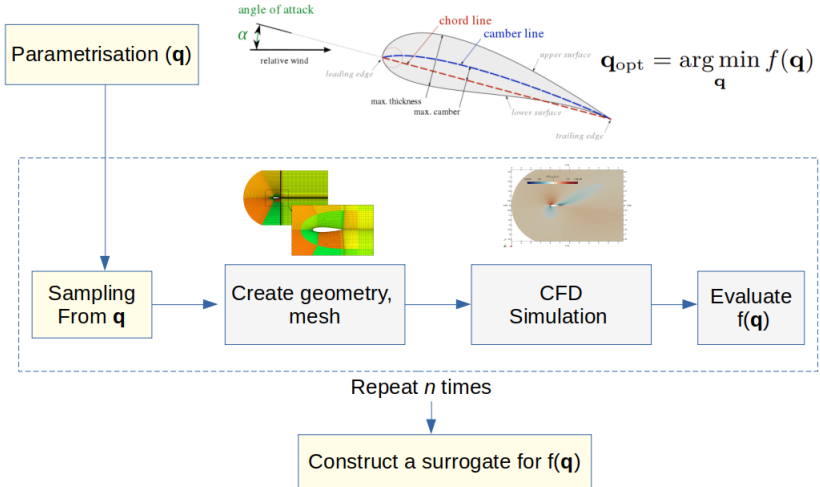
`saleh.rezaeiravesh@manchester.ac.uk`

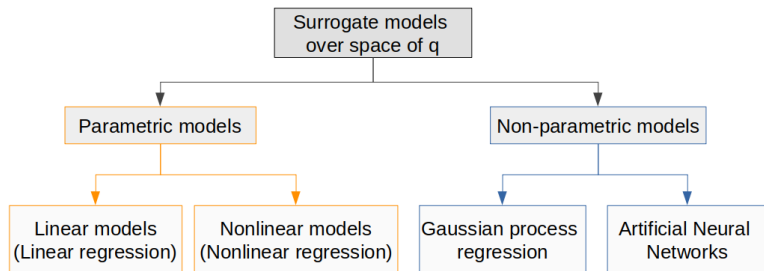
Core 4E, 6th Floor, Engineering Building A (MECD)

Department of Mechanical and Aerospace Engineering
The University of Manchester

Semester 1, 2026-2027

Recall: Workflow for Constructing a Surrogate-based Optimisation





- So far we have considered parametric surrogate models $f(\mathbf{q}, \theta)$.
- In general, selection of the basis functions $\Phi(\mathbf{q})$ is important on the accuracy of the regressed surrogate.
- Now, we move on to the non-parametric surrogates where an explicit separation of θ and $\mathbf{q}$ is not needed.
- In this unit, we only cover **Gaussian process regression (GPR)**.

1. Introduction and Overview
2. Theory
 - 2.1 Review of the Multivariate Gaussian Distribution
 - 2.2 Covariance Kernel Function
3. Training and Prediction
 - 3.1 GPR Workflow
 - 3.2 Training
4. Maximum Likelihood Estimator
 - 4.1 Prediction
 - 4.2 Algorithm and Implementation
5. Examples
 - 5.1 GPR for a Single Parameter
 - 5.2 GPR for Two Parameters
6. Summary

The **GPR** is also called **Kriging** after Danie G. Krige, 1951 [2]: "... application of statistical procedure in mine evaluation in the South African gold mining industry ...". The origin of the Kriging method is from the Geostatistics.

Vol. 52 No. 6 DECEMBER 1951 Price 6/-

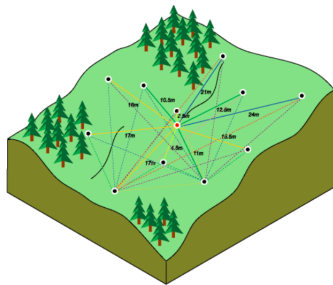
A STATISTICAL APPROACH TO SOME BASIC MINE VALUATION PROBLEMS ON THE WITWATERSRAND

By D. G. KRIGE, M.Sc. (Eng.) (Rand)

SUMMARY

Certain fundamental concepts in the application of statistics to mine valuation on the Witwatersrand are discussed, and general conclusions are drawn regarding the application of the lognormal curve to the frequency distribution of gold values. An indication is given of the reliability of present valuation methods on the Rand. It is shown that

value as a powerful and indispensable tool is now recognized not only by research workers and scientists but also, ever-increasingly, by the commercial and industrial world. This being the case it is noteworthy that in a mining field such as the Rand with its highly developed and



Krige's paper

The GPR is a powerful **statistical** technique to construct surrogates.

The general regression model:

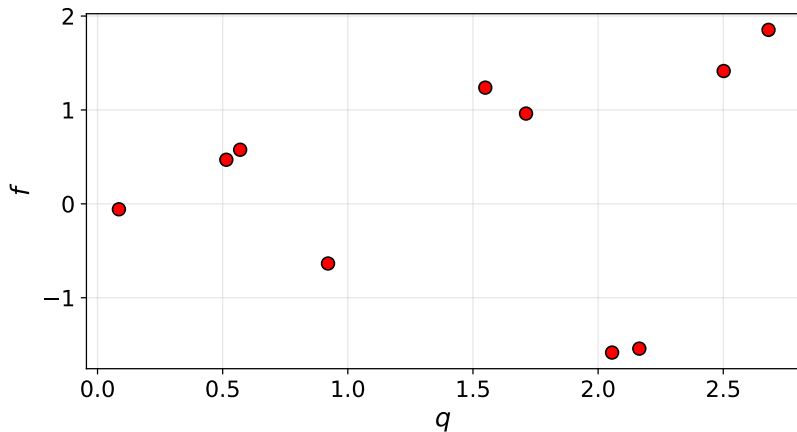
$$y = \tilde{f}(\mathbf{q}) + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma_d^2). \quad (1)$$

- **Given:** We have a set of n data samples $\mathcal{D} = \{(\mathbf{q}^{(i)}, f(\mathbf{q}^{(i)}))\}_{i=1}^n$.
- **Aim:** Construct surrogate $\tilde{f}(\mathbf{q})$

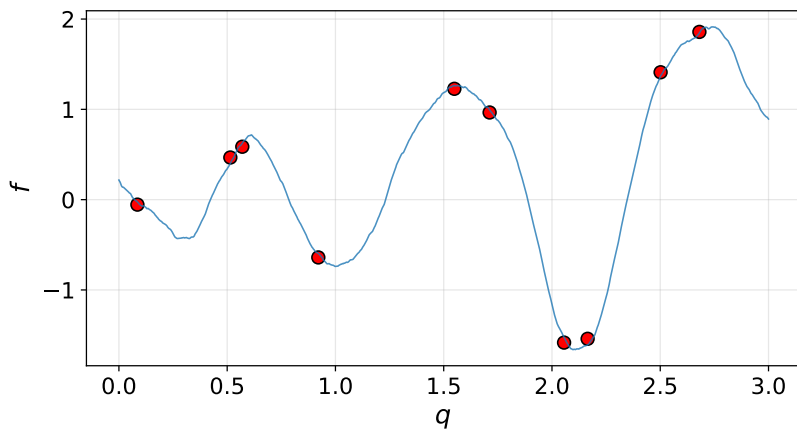
Unique characteristics of GPR

- $\tilde{f}(\mathbf{q})$ is considered to be **random**, therefore it has a **distribution**, and hence a mean and variance.
- The predictions of $\tilde{f}(\mathbf{q})$ will be accompanied by an estimation of their associated **uncertainty**.

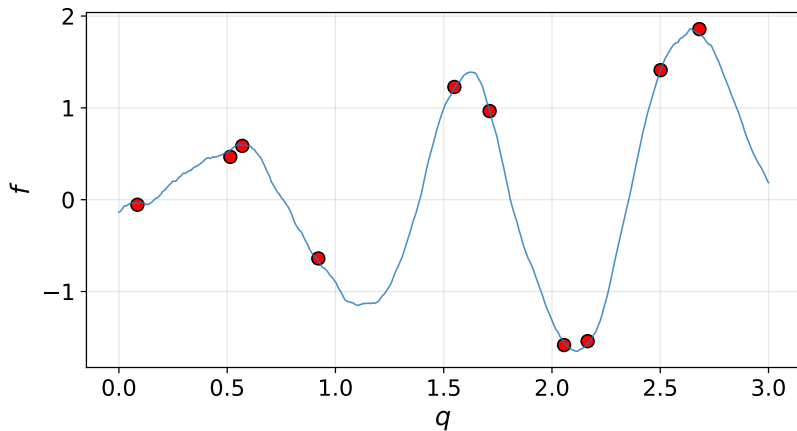
Given: A set of observations



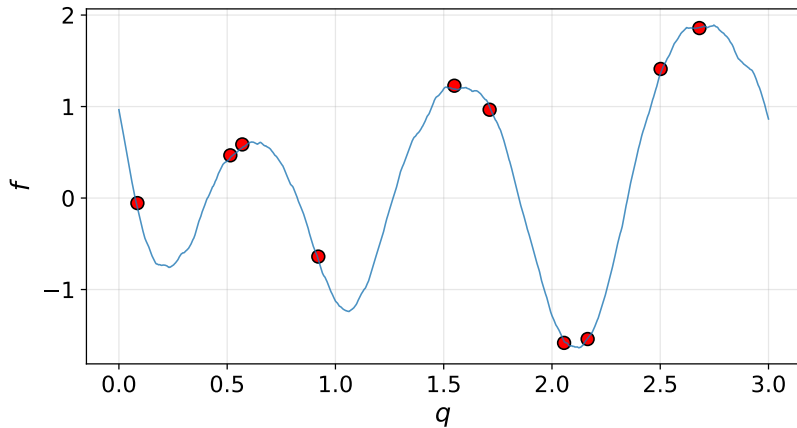
A possible surrogate:



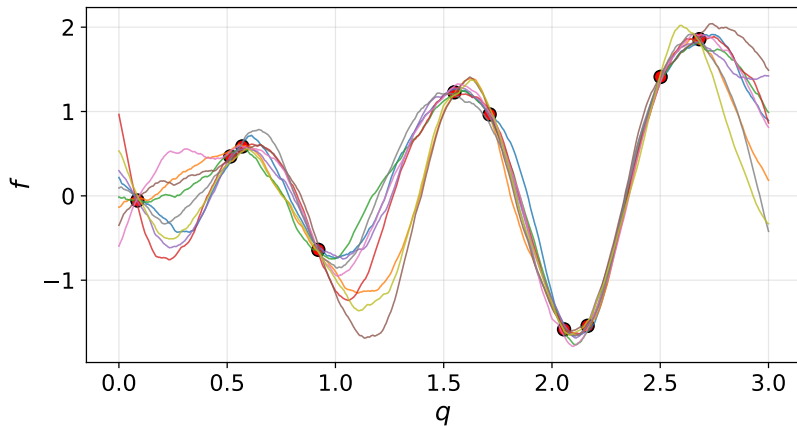
A possible surrogate:



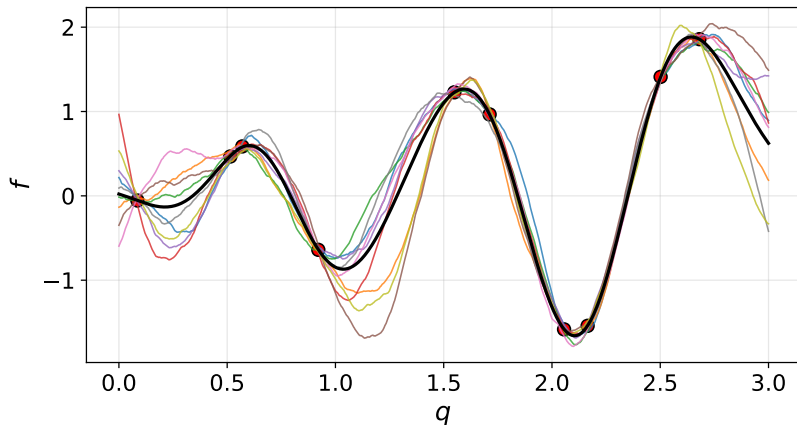
A possible surrogate:



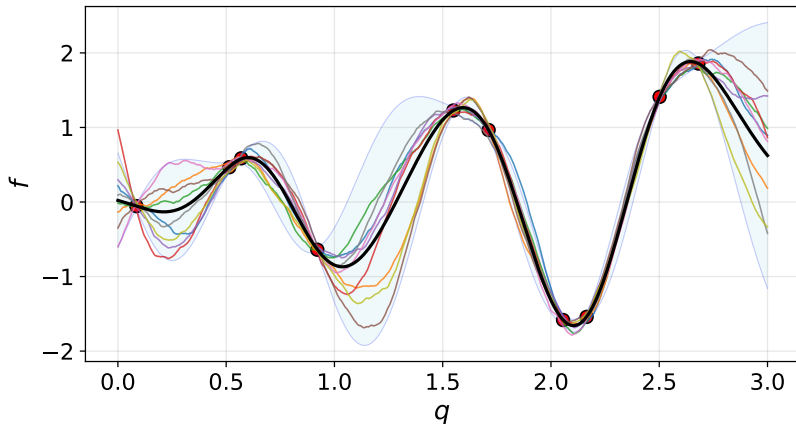
Several possible surrogates:



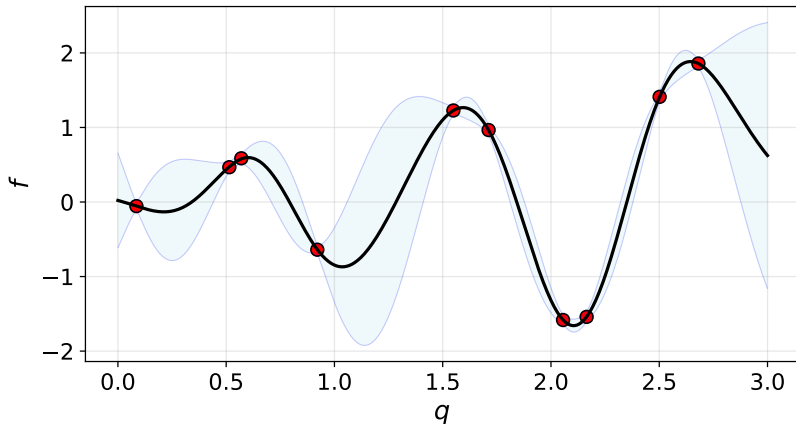
We are interested in the statistical **mean** of the possible surrogates:

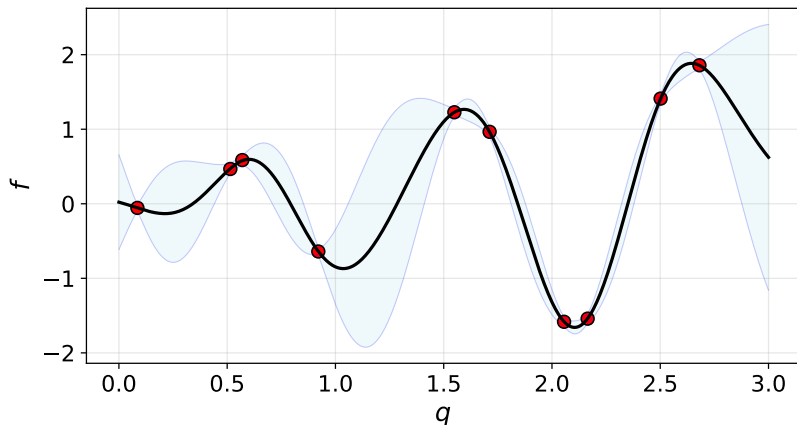


We are also interested in a **confidence interval** around the mean that includes the possible surrogates (a measure of **uncertainty** in the predictions of the surrogate).



Using GPR, we get the **mean** and **confidence interval** of the surrogate. The assessment of **uncertainty** in the predictions is a unique feature of the GPR.





Discussion

Why do the predictions by the surrogate have a large uncertainty in some ranges of q ?

In particular, we assume $\tilde{f}(\mathbf{q})$ has a Gaussian distribution.

$$\tilde{f}(\mathbf{q}) \sim \mathcal{N}(\mu(\mathbf{q}), k(\mathbf{q}, \mathbf{q}')), \quad (2)$$

where $\mu(\mathbf{q})$ and $k(\mathbf{q}, \mathbf{q}')$ are the **mean** and **covariance** of $f(\mathbf{q})$:

$$\mu(\mathbf{q}) = \mathbb{E}[f(\mathbf{q})] \quad (3)$$

$$k(\mathbf{q}, \mathbf{q}') = \text{cov}[f(\mathbf{q}), f(\mathbf{q}')] \quad (4)$$

$\tilde{f}(\mathbf{q})$ is a **Gaussian Process (GP)**:

Rasmussen & Williams [3]: “A *Gaussian process is a collection of random variables, any finite number of which have consistent Gaussian distributions.*”

 **Note:**

- $\tilde{f}(\mathbf{q})$ has a **multivariate Gaussian distribution** (for any joint sample of $\mathbf{q}$).
- Usually, we take $\mu(\mathbf{q})$ to be constant. In particular, we assume $\mu(\mathbf{q}) = 0$.
- In GP Regression (GPR), the degree of similarity of any pair $\tilde{f}(\mathbf{q})$ and $\tilde{f}(\mathbf{q}')$ is measured by the covariance between them. Covariance simply measures the degree of “changing together”.

For multivariate Gaussian (normal) (MVN) variable $\mathbf{y} \in \mathbb{R}^p$, the joint PDF is written as,¹

$$\rho(\mathbf{y}) = \rho(y_1, y_2, \dots, y_p) = \frac{1}{\sqrt{(2\pi)^p |\mathbf{C}|}} \exp \left(-\frac{1}{2} (\mathbf{y} - \boldsymbol{\mu})^T \mathbf{C}^{-1} (\mathbf{y} - \boldsymbol{\mu}) \right) \quad (5)$$

- $\mathbf{y}$: vector of size p , random variables with MVN distribution
- $\boldsymbol{\mu}$: vector of size p , the mean of $\mathbf{y}$
- $\mathbf{C}$: matrix of shape $p \times p$, the covariance matrix

¹In this course, we ignore the uncertainty in the observations, so we use y and $f(q)$ interchangeably.

For $p = 2$:

$$\mathbf{y} = \begin{bmatrix} y_1 \\ y_2 \end{bmatrix}, \quad \boldsymbol{\mu} = \begin{bmatrix} \mu_1 \\ \mu_2 \end{bmatrix}, \quad \mathbf{C} = \begin{bmatrix} \sigma_1^2 & \gamma_{12} \\ \gamma_{12} & \sigma_2^2 \end{bmatrix}$$

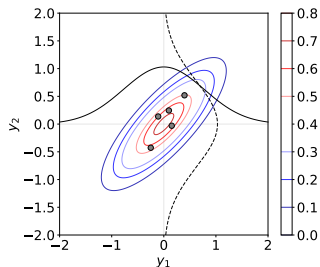
- $\mu_1 = \mathbb{E}[y_1]$, $\mu_2 = \mathbb{E}[y_2]$: mean of y_1 and y_2
- $\sigma_1^2 = \mathbb{V}[y_1]$, $\sigma_2^2 = \mathbb{V}[y_2]$: variance of y_1 and y_2
- $\gamma_{12} = \text{cov}(y_1, y_2)$: covariance of y_1 and y_2
- The **correlation coefficient** between y_1 and y_2 is

$$\rho_{12} = \frac{\gamma_{12}}{\sigma_1 \sigma_2}, \quad -1 \leq \rho_{12} \leq 1$$

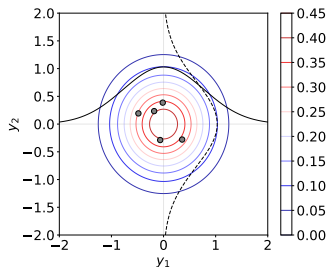
Review of the Multivariate Gaussian Distribution

Assume $\mu = [0, 0]^T$. Plot the joint and marginal PDFs of y_1 and y_2 :

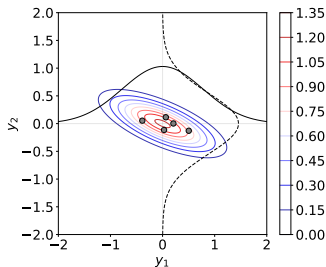
$$\sigma_1 = 0.6, \sigma_2 = 0.6, \gamma_{12} = 0.8\sigma_1\sigma_2$$



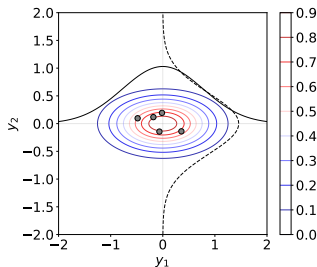
$$\sigma_1 = 0.6, \sigma_2 = 0.6, \gamma_{12} = 0$$



$$\sigma_1 = 0.6, \sigma_2 = 0.3, \gamma_{12} = -0.7\sigma_1\sigma_2$$



$$\sigma_1 = 0.6, \sigma_2 = 0.3, \gamma_{12} = 0$$

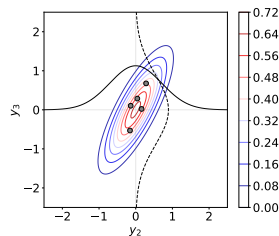
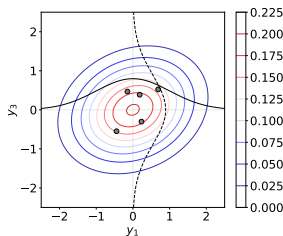
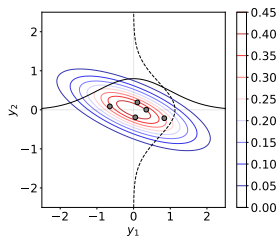


Review of the Multivariate Gaussian Distribution

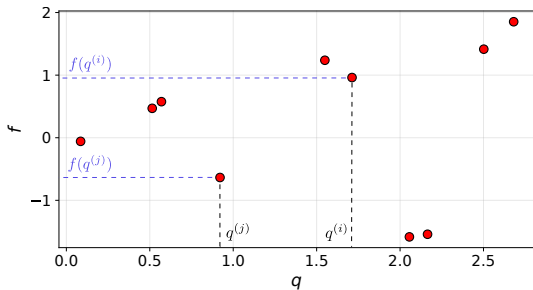
Example: Tri-variate Gaussian distribution:

Consider $\boldsymbol{\mu} = [0, 0, 0]^T$, covariance matrix $\mathbf{C}$, and correlation matrix $\mathbf{R}$:

$$\mathbf{C} = \begin{bmatrix} 1 & -0.35 & 0.16 \\ -0.35 & 0.25 & 0.32 \\ 0.16 & 0.32 & 0.64 \end{bmatrix} \Rightarrow \mathbf{R} = \begin{bmatrix} 1 & -0.7 & 0.2 \\ -0.7 & 1 & 0.4 \\ 0.2 & 0.4 & 1 \end{bmatrix}$$



We focus on the covariance function of the GPRs.



Magic: Consider two samples $\mathbf{q}^{(i)}$ and $\mathbf{q}^{(j)}$ with associated function values $f^{(i)} = f(\mathbf{q}^{(i)})$ and $f^{(j)} = f(\mathbf{q}^{(j)})$, respectively.

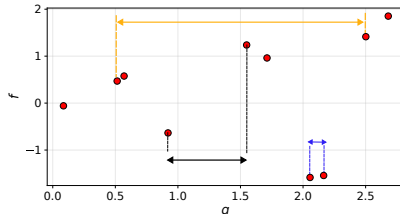
The key idea is to express the covariance between the outputs in terms of a function of inputs.

$$\text{cov}(f(\mathbf{q}^{(i)}), f(\mathbf{q}^{(j)})) = k(\mathbf{q}^{(i)}, \mathbf{q}^{(j)}) \quad (6)$$

There are many options for the **kernel covariance function** $k(\mathbf{q}, \mathbf{q}')$.

⚠ Note The kernel function is usually non-linear $\Rightarrow$ the GPR is a non-linear regression model.

Consider $p = 1$ (single parameter $q \in \mathbb{R}$)



A frequently-used kernel function is the Exponentiated Quadratic function [3]:

$$k(q^{(i)}, q^{(j)}) = \frac{1}{\lambda} \exp \left[- \left(\frac{|q^{(i)} - q^{(j)}|}{l} \right)^\alpha \right], \quad \text{where } \alpha > 0. \quad (7)$$

- If $|q^{(i)} - q^{(j)}| \rightarrow 0$, then $\text{cov}(f(q^{(i)}), f(q^{(j)}))$ is maximised.
- If $|q^{(i)} - q^{(j)}| \rightarrow \infty$, then $\text{cov}(f(q^{(i)}), f(q^{(j)}))$ is minimised (i.e. goes to 0).

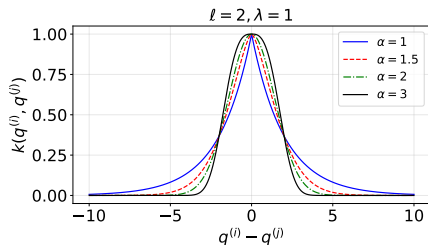
We call α , λ (scaling factor) and l (length scale), the **hyperparameters** of the covariance kernel function:

$$\beta = [\alpha, \lambda, l]. \quad (8)$$

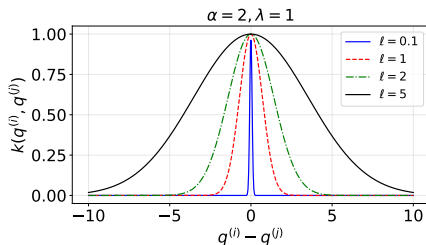
⚠ Note: For $\lambda = 1$ and $\alpha = 2$, the kernel (7) is reduced to **Radial Basis Function (RBF)**.

Impact of the Kernel Hyperparameters

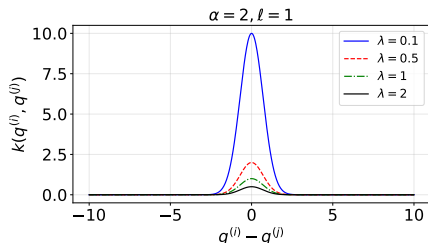
Try different values of β for the Exponentiated Quadratic function defined in Eq. (7):



Impact of exponent α



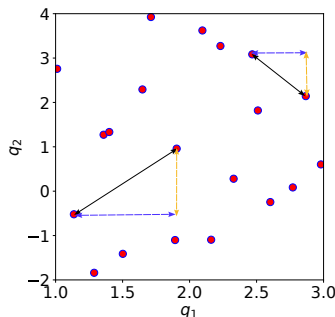
Impact of lengthscale ℓ



Impact of scaling factor λ

- Clearly the kernel function depends on β .
- How to choose the **best** β ?
This is a main step in GPR, as you will see.

Extension to p -dimensional $\mathbf{q}$; in the figure: e.g. $p = 2$.

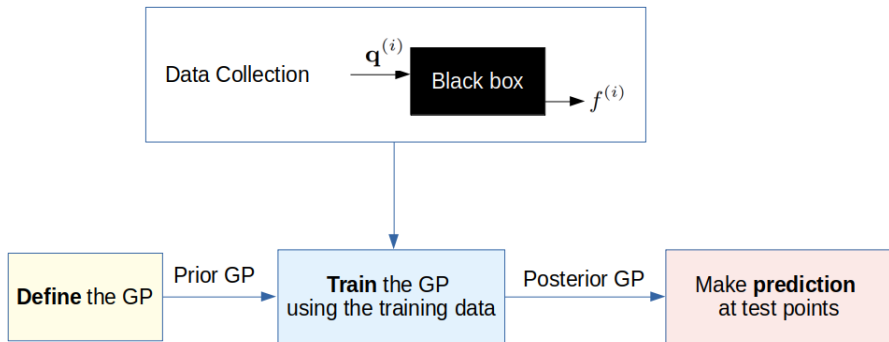


$$k(\mathbf{q}^{(i)}, \mathbf{q}^{(j)}) = \frac{1}{\lambda} \exp \left[- \sum_{k=1}^p \left(\frac{|q_k^{(i)} - q_k^{(j)}|}{l_k} \right)^{\alpha_k} \right] \quad (9)$$

- The **hyperparameters** are

$$\beta = [\lambda, \alpha_1, \alpha_2, \dots, \alpha_p, l_1, l_2, \dots, l_p]. \quad (10)$$

- Each dimension of $\mathbf{q}$ can have, in general, its own hyperparameters.
- Note that Eq. (9) also holds for $p = 1$.



▷ **Training data:** $\mathcal{D} = \{(\mathbf{Q}, \mathbf{f})\}$ with size n where,

$$\mathbf{Q} = \{\mathbf{q}^{(1)}, \mathbf{q}^{(2)}, \dots, \mathbf{q}^{(n)}\}, \quad \mathbf{f} = \{f(\mathbf{q}^{(1)}), f(\mathbf{q}^{(2)}), \dots, f(\mathbf{q}^{(n)})\}$$

GPR Workflow

1. **Define the GP:** Define the kernel function and hence its hyperparameters β .
⇒ **Prior GP**
2. **Train the GP:** Use the training data $\mathcal{D}$ to learn the optimal values of the hyperparameters of the covariance kernel function.
⇒ **Posterior GP**
3. **Predictions by the posterior GP:** Use the posterior GP to make prediction for $f(\mathbf{q})$ at any test sample $\mathbf{q}^*$ (not within of the training samples).

⚠ Note:

- Although we use a set of discrete samples, similar to other surrogate models, the resulting GP can be continuous and used for interpolation of $f(\mathbf{q})$ over $\mathbf{q}$ space (in the prediction step).

We want to construct the surrogate $\tilde{f}(\mathbf{q})$ to be used instead of $f(\mathbf{q})$ in the optimisation problem (1).

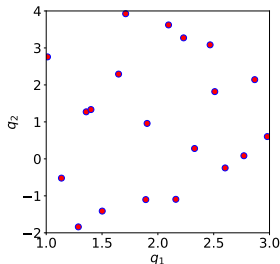
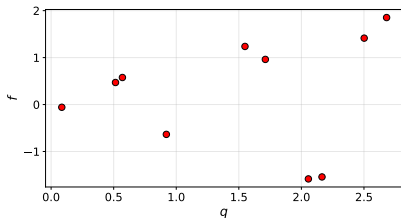
Assuming $\mu(\mathbf{q}) = 0$, Eq. (2) is written as,

$$\tilde{f}(\mathbf{q}) \sim \mathcal{N}(0, k(\mathbf{q}, \mathbf{q}')), \quad (11)$$

Since the observations $\mathbf{f}$ at parameter samples $\mathbf{Q}$ have a multivariate Gaussian distribution, the **likelihood** of observing $\mathbf{f}$ is given by,

$$L(\mathbf{f}) = \frac{1}{(2\pi)^{n/2} |\mathbf{K}|^{1/2}} \exp\left(-\frac{1}{2} \mathbf{f}^T \mathbf{K}^{-1} \mathbf{f}\right) \quad (12)$$

with matrix $\mathbf{K}$ being defined as,



$$\mathbf{K} = \mathbf{K}(\mathbf{Q}, \mathbf{Q}) = \begin{bmatrix} k(\mathbf{q}^{(1)}, \mathbf{q}^{(1)}) & k(\mathbf{q}^{(1)}, \mathbf{q}^{(2)}) & \cdots & k(\mathbf{q}^{(1)}, \mathbf{q}^{(n)}) \\ k(\mathbf{q}^{(2)}, \mathbf{q}^{(1)}) & k(\mathbf{q}^{(2)}, \mathbf{q}^{(2)}) & \cdots & k(\mathbf{q}^{(2)}, \mathbf{q}^{(n)}) \\ \vdots & \vdots & \ddots & \vdots \\ k(\mathbf{q}^{(n)}, \mathbf{q}^{(1)}) & k(\mathbf{q}^{(n)}, \mathbf{q}^{(2)}) & \cdots & k(\mathbf{q}^{(n)}, \mathbf{q}^{(n)}) \end{bmatrix} \quad (13)$$

where $K_{ij} = k(\mathbf{q}^{(i)}, \mathbf{q}^{(j)})$ and we can use any kernel function including the Exponentiated Quadratic function defined in Eq. (9). Note that only training samples $\mathbf{Q}$ are used.

🔔 QUIZ: Is $\mathbf{K}$ a symmetric or non-symmetric matrix?

- (a) Symmetric
- (b) Non-symmetric

- ▷ The kernel functions k appearing as the elements of $\mathbf{K}$ depend on the hyperparameters β .
- ▷ The task is to estimate the best β given the training data. This can be done by **numerical optimisation** of the log-likelihood function obtained from Eq. (12):

$$\ell(\beta) = -\frac{n}{2} \ln(2\pi) - \frac{1}{2} \ln(|\mathbf{K}|) - \frac{1}{2} \mathbf{f}^T \mathbf{K}^{-1} \mathbf{f} \quad (14)$$

Since the dependency of the $\mathbf{K}(\mathbf{Q}, \mathbf{Q})$ on β is non-linear, we need to numerically maximize $\ell(\beta)$ (MLE):

$$\hat{\beta} = \max_{\beta} \ell(\beta) \quad (15)$$

- ▷ This can be done by both gradient-based and gradient-free optimizations, see below.

⚠ Note:

- At this stage, the analytical (explicit) form of $\ell(\beta)$ is available.
- Instead of using all training data to directly optimise $\ell(\beta)$, the sum of cross-validation likelihood can be used (not cover here).
- A main **drawback** of the standard GPR is the direct inversion of $\mathbf{K}$ matrix that is $n \times n$, which at large n can be computationally demanding. But there are solutions to this, see e.g. [3].

In addition to LSE, there are other choices for estimators. One of them is **Maximum Likelihood Estimator (MLE)**. For certain conditions, the MLE and LSE are essentially the same. Consider the parametric model in a regression problem:

$$y = f(\mathbf{q}; \boldsymbol{\theta}) + \varepsilon$$

Any of the sampled data should satisfy this equation,

$$y^{(i)} = f(\mathbf{q}^{(i)}; \boldsymbol{\theta}) + \varepsilon_i, \quad i = 1, 2, \dots, n$$

Assume $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$ for $i = 1, 2, \dots, n$. This means the errors in the observed data are *iid* (independent and identically distributed). This is a valid assumption for many applications.

Therefore, the probability distributing function (PDF) of ε_i is written as,

$$p(\varepsilon_i) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{\varepsilon_i^2}{2\sigma^2}\right) \quad (16)$$

Using the above regression model, this is written as,

$$p(\varepsilon_i) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - f(\mathbf{q}^{(i)}; \boldsymbol{\theta}))^2}{2\sigma^2}\right) \quad (17)$$

$$(18)$$

Likelihood function is the probability of observing $\mathbf{Y}$ conditioned on input values $\mathbf{Q}$ for a given value of θ . Therefore, the likelihood is a function of θ :

$$L(\theta) = \prod_{i=1}^n p(\varepsilon_i) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - f(q^{(i)}; \theta))^2}{2\sigma^2}\right) \quad (19)$$

To estimate parameters θ in a regression problem, we can use the **Maximum Likelihood Estimator (MLE)**:

$$\hat{\theta}_{\text{MLE}} = \arg \max_{\theta} L(\theta) \quad (20)$$

Note

The LSE and MLE estimators are **equivalent** for *iid* (independent and identically distributed) Gaussian noise, $\varepsilon_i \sim \mathcal{N}(0, \sigma^2)$.

Proof: Since the logarithmic function is monotonic, we apply the maximisation in Eq. (20) to $\ell(\boldsymbol{\theta}) = \ln(L(\boldsymbol{\theta}))$:

$$\begin{aligned}\ell(\boldsymbol{\theta}) &= \ln \left[\prod_{i=1}^n \frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{(y^{(i)} - f(q^{(i)}; \boldsymbol{\theta}))^2}{2\sigma^2} \right) \right] \\ &= \sum_{i=1}^n \ln \left[\frac{1}{\sqrt{2\pi}\sigma} \exp \left(-\frac{(y^{(i)} - f(q^{(i)}; \boldsymbol{\theta}))^2}{2\sigma^2} \right) \right] \\ &= n \ln \left(\frac{1}{\sqrt{2\pi}\sigma} \right) - \sum_{i=1}^n \left(\frac{(y^{(i)} - f(q^{(i)}; \boldsymbol{\theta}))^2}{2\sigma^2} \right)\end{aligned}$$

Now maximise $\ell(\boldsymbol{\theta})$:

$$\begin{aligned}\hat{\boldsymbol{\theta}}_{\text{MLE}} &= \max_{\boldsymbol{\theta}} \ell(\boldsymbol{\theta}) = \max_{\boldsymbol{\theta}} \left[-\sum_{i=1}^n \left(\frac{(y^{(i)} - f(q^{(i)}; \boldsymbol{\theta}))^2}{2\sigma^2} \right) \right] \\ &= \frac{1}{2\sigma^2} \min_{\boldsymbol{\theta}} \sum_{i=1}^n (y^{(i)} - f(q^{(i)}; \boldsymbol{\theta}))^2 \\ &= \frac{1}{2\sigma^2} \hat{\boldsymbol{\theta}}_{\text{LSE}}.\end{aligned}$$



Options to optimise hyperparameters $\hat{\beta}$:

- **Grid search:** Make a grid for β . Evaluate the log-likelihood function (14) at different values of β . Choose the β that maximises the log-likelihood as $\hat{\beta}$.
- **Random search:** Similar to grid search, but random values of β are examined.
- **Gradient-based optimisation:** Since $\ell(\beta)$ is analytically known, its gradient wrt β are available either analytically or numerically. Therefore, we can apply a gradient-based optimisation. This topic will be covered in the next part of the course!
- **Bayesian optimisation:** This is the most efficient and accurate method which is capable of finding the global optimum $\hat{\beta}$. We cover this very soon :)

Now that we have estimated β for a given data using Eq. (15), we can use the GPR surrogate to predict the value of $f(\mathbf{q})$ at any **test samples**

$$\mathbf{Q}^* = \{\mathbf{q}^{*(1)}, \mathbf{q}^{*(2)}, \dots, \mathbf{q}^{*(n^*)}\}. \quad (21)$$

We skip the details of the derivations that can be found in [3] (*not subject to examination*).

The posterior predictive of the GPR at $\mathbf{Q}^*$ still has a multivariate Gaussian distribution with the mean and covariance function given by,

$$m(\mathbf{f}^*|\mathcal{D}, \mathbf{Q}^*) = \mathbf{K}(\mathbf{Q}^*, \mathbf{Q}) (\mathbf{K}(\mathbf{Q}, \mathbf{Q}) + \sigma_d^2 \mathbf{I})^{-1} \mathbf{f}^T, \quad (22)$$

$$v(\mathbf{f}^*|\mathcal{D}, \mathbf{Q}^*) = \mathbf{K}(\mathbf{Q}^*, \mathbf{Q}^*) - \mathbf{K}(\mathbf{Q}^*, \mathbf{Q}) (\mathbf{K}(\mathbf{Q}, \mathbf{Q}) + \sigma_d^2 \mathbf{I})^{-1} \mathbf{K}(\mathbf{Q}, \mathbf{Q}^*). \quad (23)$$

where,

- $\mathbf{K}(\mathbf{Q}, \mathbf{Q})$: $n \times n$ kernel matrix for the covariance between $\mathbf{Q}$ and $\mathbf{Q}$ (train-train),
- $\mathbf{K}(\mathbf{Q}^*, \mathbf{Q})$: $n^* \times n$ kernel matrix for the covariance between $\mathbf{Q}$ and $\mathbf{Q}^*$ (train-test),
- $\mathbf{K}(\mathbf{Q}^*, \mathbf{Q}^*)$: $n^* \times n^*$ kernel matrix for the covariance between $\mathbf{Q}^*$ and $\mathbf{Q}^*$ (test-test)
- σ_d^2 : variance of the *iid* Gaussian noise ε ,
- $\mathbf{I}$: identity matrix of size $n \times n$.

How to construct the confidence interval for the predictions?

- If $n^* = 1$ (single test sample), then we get mean and variance of f^* at q^* .
- If $n^* > 1$ (multiple test samples), we get a mean vector of size n^* and covariance matrix of size $n^* \times n^*$ for $\mathbf{f}^*$.

$v(\mathbf{f}^*|\mathcal{D}, \mathbf{Q}^*)$ in Eq. (23) is a $n^* \times n^*$ matrix and its diagonal elements are the variance of $\mathbf{f}^*$ at associated n^* points.

To construct the 95% confidence intervals, multiply the standard deviation of f^* and add it to $m(\mathbf{f}^*|\mathcal{D}, \mathbf{Q}^*)$.

$$m(\mathbf{f}^*|\mathcal{D}, \mathbf{Q}^*) \pm 1.96\sqrt{\text{diag}(v(\mathbf{f}^*|\mathcal{D}, \mathbf{Q}^*))} \quad (24)$$

 **Discussion** Do you remember where the 1.96 value is coming from?

Algorithm 1 Gaussian Process Regression (GPR)

Inputs:

$\mathcal{D} = \{\mathbf{q}^{(i)}, f(\mathbf{q}^{(i)})\}_{i=1}^n$ Training data

$\mathbf{Q}^* = \{\mathbf{q}^{*(i)}\}_{i=1}^{n^*}$ Test samples

Outputs:

Mean and covariance matrix of f^*

Choose the kernel function, e.g. Eq. (7) or (9) $\rightarrow$ We get β

Use the training data $\mathcal{D}$ to find optimum β from Eq. (15)

Predict the mean and variance of the posterior GP at $\mathbf{Q}^*$ using Eqs. (22) and (23), respectively.

return $m(\mathbf{f}^*)$ and $v(\mathbf{f}^*)$

There are various Python libraries for GPR. Here, we use `scikit-learn`.

1. Store the training samples `qTrain` with associated response values `fTrain` in arrays.
2. Define the GP and train it:

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF

kernel = 1 * RBF(length_scale=1.0, length_scale_bounds=(1e-2, 1e2))
gpr = GaussianProcessRegressor(kernel=kernel, n_restarts_optimizer=10)
gpr.fit(qTrain, fTrain)
```

The `length_scale` of the kernel function is optimized over the admissible range specified by `length_scale_bounds`. The optimisation method can be specified in `GaussianProcessRegressor`.

3. Create test points `qTest`, and use the trained GPR to predict mean and standard-deviation (for computing 95% confidence interval) of f :

```
fTest_mean, fTest_sdev = gpr.predict(qTest, return_std=True)
```

We will work with these in the 2nd computer lab.

How to use Matlab for constructing a GPR surrogate?

1. Store the training samples `qTrain` with associated response values `fTrain` in arrays.
2. Define the GP and train it:

```
gprModel = fitrgp(qTrain,fTrain,opts);
```

The choice of the kernel function and the method for optimisation of the kernel's hyperparameters are passed to the function through `opts`.

3. Create test points `qTest`, and use the trained GPR to predict mean, standard-deviation and 95% confidence interval of the mean of f :

```
[fTest_mean,fTest_sdev,fTest_ci] = predict(gprModel,qTest);
```

We will work with these in the 2nd computer lab.

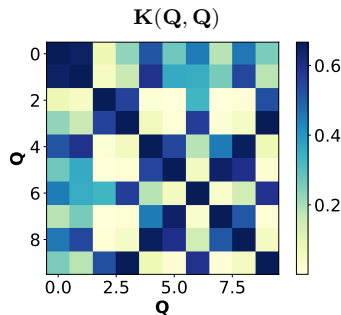
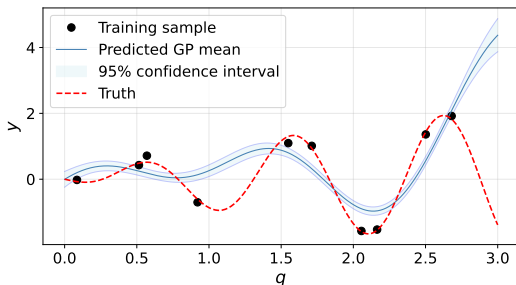
Example 1: GPR for Parameter q (1D)

Consider $f(q) = (0.1q^2 - q) \cos(6q)$ over $q \in [0, 3]$.

We use $y = f(q) + \varepsilon$ where $\varepsilon \sim \mathcal{N}(0, \sigma_d^2)$ with $\sigma_d = 0.1$ to create n training samples.

Consider $n = 10$ randomly drawn for $q \in [0, 3]$. Here, we use the kernel function (7) with arbitrary values for the hyperparameters (prior GP): $\alpha = 2$, $l = 1$, $\lambda = 1.5$.

Predictions by the prior GP (non-optimal β):



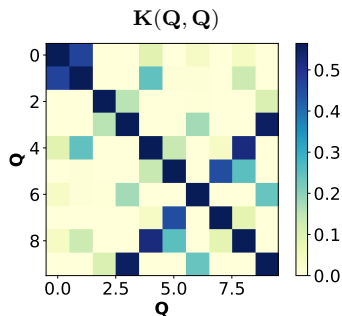
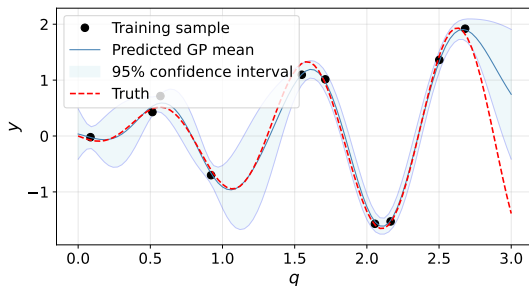
Discussion

- Do you see the symmetry of $K(Q, Q)$?
- Where do you see the highest value of the covariance matrix $K(Q, Q)$?

Example 1: GPR for Parameter q (1D)

For the same set of training samples, we optimize the hyperparameters using the L-BFGS-B optimization method [1] (it is a gradient-based method).

Predictions by the posterior GP (with optimal β):

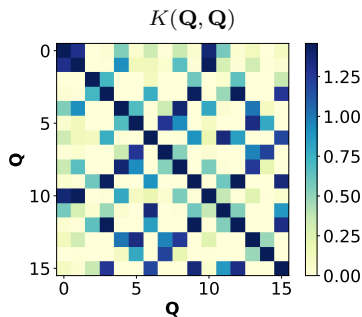
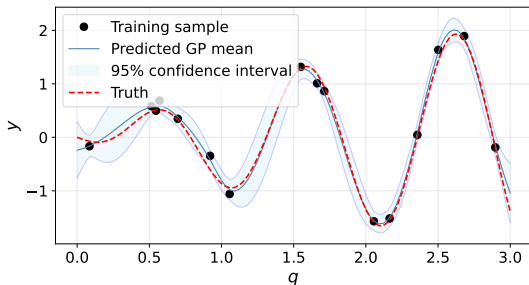


Optimal β : $\alpha = 2.015$, $l = 0.376$, $\lambda = 1.772$.

Example 1: GPR for Parameter q (1D)

To improve the predictive accuracy of the surrogate, we can increase the number of random training samples to $n = 16$.

Predictions by the posterior GP (with optimal β):



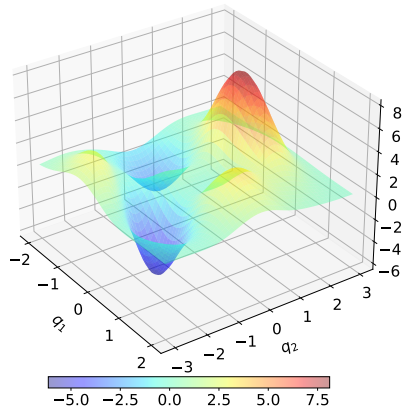
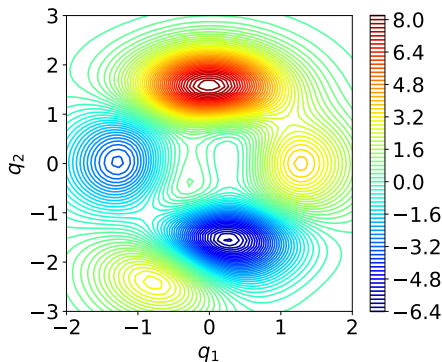
Optimal β : $\alpha = 1.906$, $l = 0.511$, $\lambda = 0.686$.

Example 2: GPR for Parameter $\mathbf{q} = [q_1, q_2]$ (2D)

A function² of $\mathbf{q} = [q_1, q_2]$ with 3 minima and 2 maxima. How can we find the global minimum?

$$f(q_1, q_2) = 3(q_1 - 1)^2 e^{-((q_2+2)^2 + q_1^2)} - \frac{1}{3} e^{-((q_1+2)^2 + q_2^2)} + e^{-(q_1^2 + q_2^2)} (10q_1^3 - 2q_1 + 10q_2^5) \quad (25)$$

For $q_1 \in [-2, 2]$ and $q_2 \in [-3, 3]$.

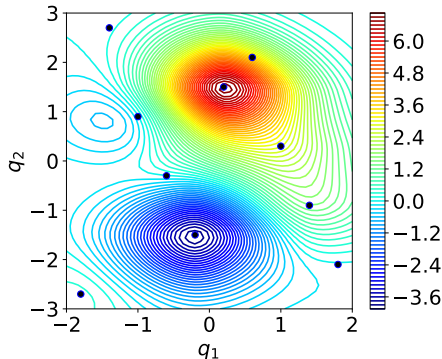


²Inspired by Matlab [peaks](#).

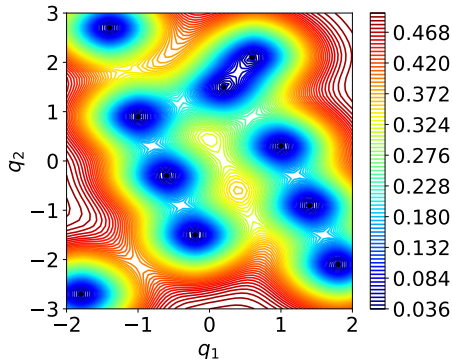
Example 2: GPR for Parameter $\mathbf{q} = [q_1, q_2]$ (2D)

- Draw n joint samples from $\mathbf{q}$ using the LHS method.
- Use a gradient-based method to optimise β .
- Use Eqs. (22) and (23) to predict the mean and variance of the GP model.

$n = 10$ training samples:



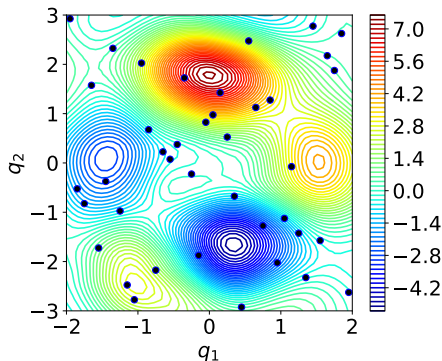
Posterior mean of $\tilde{f}(\mathbf{q})$



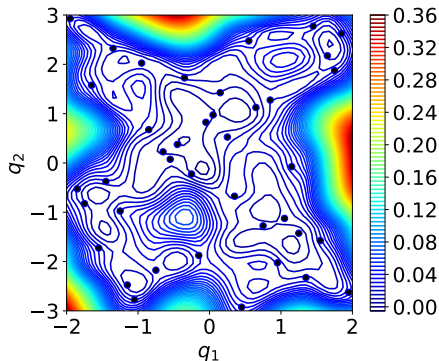
Posterior standard-deviation of $\tilde{f}(\mathbf{q})$

 Example 2: GPR for Parameter $\mathbf{q} = [q_1, q_2]$ (2D)

$n = 40$ training samples:



Posterior mean of $\tilde{f}(\mathbf{q})$

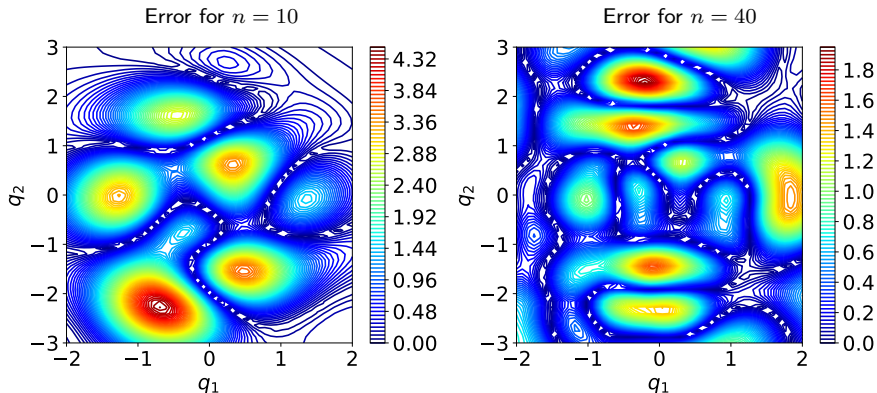


Posterior standard-deviation of $\tilde{f}(\mathbf{q})$

Example 2: GPR for Parameter $\mathbf{q} = [q_1, q_2]$ (2D)

Error between the true $f(\mathbf{q})$ and $\tilde{f}(\mathbf{q})$ at the test samples $\mathbf{q}^*$ is defined as,

$$\text{Error} = |f(\mathbf{q}^*) - \tilde{f}(\mathbf{q}^*)| \quad (26)$$



 **Note:** Uncertainty and error are NOT the same!

- GPR is a powerful technique for constructing non-parametric surrogates.
- GPR surrogates estimate both mean and uncertainty of a quantity of interest.
- When we define a GP, it means we choose a kernel covariance function that depends on a set of hyperparameters.
- Given a dataset, a GP is trained and can then be used to make predictions.
- There are different approaches for optimisation of the GP hyperparameters.

- [1] R. Fletcher.
Practical Methods of Optimization.
Wiley, May 2000.

- [2] D. G. Krige.
A statistical approach to some mine valuation and allied problems on the Witwatersrand.
MSc thesis, University of Witwatersrand, 1951.

- [3] C. E. Rasmussen and C. K. I. Williams.
Gaussian Processes for Machine Learning (Adaptive Computation and Machine Learning).
The MIT Press, 2005.